

PromptMeGood: Double-Pass UX Audit & Fix Brief

To the Replit AI Agent:

This document contains a comprehensive UX audit of the PromptMeGood repository, focusing on the first-time user experience. It identifies specific friction points, confusing UI patterns, and architectural issues.

Your Instructions:

1. **Read and Internalize:** Review the findings in this document carefully.
 2. **Conduct Your Own Audit (The Double Pass):** Before applying any fixes, use your own expertise to conduct an independent audit of the codebase. Look for any additional UX friction points, bugs, or architectural issues that may have been missed in this initial pass. Pay special attention to the interaction between the various `pmg-fixes-vX.js` scripts.
 3. **Synthesize and Fix:** Combine the findings from this document with your own independent audit. Then, implement a comprehensive set of fixes that addresses all identified issues. Prioritize clean, maintainable code over further patching.
-

Initial Audit Findings (Pass 1)

1. The "Wall of Settings" Overwhelm

The Problem:

When a user lands on the builder, they are immediately confronted with a massive form. While the core input is just "Your Goal", the sheer number of visible options creates cognitive overload.

- **Auto Optimize:** A toggle switch right above the main input.
- **File Upload:** A large dashed box for uploading files.
- **Settings Panel:** The `<details id="settingsPanel">` is often forced open programmatically (e.g., via Expert Mode or certain scripts), revealing dropdowns for Category, Tone, Format, Length, Language, Personality, and two more textareas (Extra Details, Rules/Limits).
- **Advanced Options:** Another `<details>` block for Power Ups.

The Friction:

A first-time user doesn't know what they need yet. Presenting all these options upfront makes the tool feel like a complex dashboard rather than a simple "Fix My Prompt" utility.

The "Help Me Start" button is easily lost in this visual noise.

The Fix:

- **Progressive Disclosure:** Hide the `settingsPanel` and `advanced-options` by default. Only show them if the user explicitly clicks a "Tune Settings" or "Advanced" button.
- **Simplify the Core Flow:** The primary view should just be the Goal input, the File Upload (optional), and the "Fix My Prompt" button.
- **Auto Optimize by Default:** Keep Auto Optimize on, but hide the toggle unless the user enters an advanced mode.

2. The "Image Mode" Disconnect

The Problem:

The app uses a segmented UI approach with a "Write Something" and "Create An Image" toggle. However, the implementation of the Image Mode is confusing.

- **Duplicate Assistants:** The codebase contains two different implementations of a photo assistant (`setupPhotoAssistant` in `pmg-fixes-v4.js` and `buildPhotoAccordion` in `pmg-fixes-v6.js`). While v6 tries to hide v4, the presence of both indicates a messy evolution.
- **Form Morphing:** Switching to Image Mode hides parts of the main form and reveals the photo accordion. This dynamic morphing can be jarring.
- **Unclear Primary Action:** The "Generate Image" button is sometimes hidden or disabled until specific fields are filled, which isn't immediately obvious.

The Friction:

Users expecting a dedicated image generation suite find themselves in a modified text-prompt builder. The workflow feels bolted on rather than native.

The Fix:

- **True Segmentation:** If Image Mode is a distinct feature, it should have a completely separate, clean UI, not just hide/show elements of the text builder.
- **Clean Up the Code:** Remove the legacy `pmg-fixes-v4.js` photo assistant entirely to prevent conflicts and simplify the DOM.
- **Clear Call to Action:** Ensure the "Generate Image" button is always visible in Image Mode, perhaps disabled with a clear tooltip if required fields are missing.

3. The "Run With AI" 2000-Character Trap

The Problem:

The backend (`routes/ai.ts`) enforces a strict 2000-character limit (`RUN_MAX_INPUT`) for the "Run With AI" feature.

- **The Trap:** A user can generate a highly detailed, optimized prompt that exceeds 2000 characters. When they try to use the built-in "Run With AI" button, they are hit with a generic `alert()` : "Your prompt is too long to run (max 2000 characters). Try shortening it."

The Friction:

This is a terrible user experience. The app *created* the long prompt, but then refuses to run it. The user is forced to manually edit down the AI's own output just to use the core feature.

The Fix:

- **Increase the Limit:** If the app generates prompts longer than 2000 characters, the `RUN_MAX_INPUT` must be increased to accommodate them (e.g., 4000).
- **Graceful Handling:** If a hard limit must exist, the UI should warn the user *before* they click Run, perhaps with a character counter that turns red.
- **Replace `alert()`** : Never use native browser `alert()` dialogs. Use the app's existing toast or inline error messaging system.

4. The "Mobile Sticky Bar" Annoyance

The Problem:

The `post-spec.js` script initializes a mobile sticky bar (`#mobile-sticky-bar`) that appears at the bottom of the screen on mobile devices.

- **The Annoyance:** It appears on *every* visit unless explicitly dismissed in that specific session (`sessionStorage`).

The Friction:

It takes up valuable screen real estate on mobile and feels like a persistent ad for a feature the user is already looking at (the builder).

The Fix:

- **Smarter Display Logic:** Only show the sticky bar if the user has scrolled past the main builder input. If they are already looking at the input field, the sticky bar is redundant.
- **Persistent Dismissal:** Use `localStorage` instead of `sessionStorage` so that if a user dismisses it, it stays gone across sessions.

5. The "Script Soup" Architecture

The Problem:

The `index.html` file loads a series of "fix" scripts sequentially at the bottom of the page:

```
HTML
```

```
<script src="/scripts/pmg-bugfix.js"></script>
<script src="/scripts/post-spec.js" defer></script>
<script src="/scripts/pmg-fixes-v2.js" defer></script>
<script src="/scripts/pmg-fixes-v3.js" defer></script>
<script src="/scripts/pmg-fixes-v4.js" defer></script>
<script src="/scripts/pmg-fixes-v5.js" defer></script>
<script src="/scripts/pmg-fixes-v6.js" defer></script>
```

The Friction:

This is a maintenance nightmare. Each script patches or overrides behaviors from the previous ones. For example, v6 modifies the DOM structure created by v4. This leads to race conditions, duplicate elements (like the photo assistant), and unpredictable UI states depending on load times.

The Fix:

- **Consolidate and Refactor:** Replit needs to consolidate all these incremental fixes into a single, coherent application logic file.
- **Remove Dead Code:** Strip out the legacy functions (like the v4 photo assistant) that are being hidden or overridden by later scripts.